

Scalable Document Reading with LLM-Augmented Navigation

Scalable Document Reading with LLM-Augmented Navigation

A. R. Ciment and D. A. Cask

Department of Computer Science, University of Imaginary Sciences, Dreamland.

Abstract. We present a lightweight architecture for browser-based scientific PDF readers that combines high-fidelity canvas rendering with large language model assistance. Our system renders pages using PDF.js, synchronizes a text-layer for selectable content, and exposes an AI side panel for contextual summarization and question answering. We evaluate rendering latency across document lengths up to one hundred pages and show that the proposed approach maintains interactive frame rates. A user study with thirty-two participants indicates that integrated navigation and conversational explanations reduce the time required to locate key sections by approximately forty percent.

1 Introduction

Reading long technical documents in the browser remains a frustrating experience. Users frequently switch between a PDF viewer, a note-taking application, and a separate chat interface to ask questions about the content. Existing web-based readers either render documents faithfully but offer no assistance, or they extract plain text and lose the original typography and figures.

We introduce Cimedaca Reader, a single-page application built on SvelteKit that marries faithful page rendering with a conversational AI companion. The reader loads the Portable Document Format binary in the browser, renders each page to a canvas, overlays a transparent text layer for selection and search, and displays an AI panel on the right side of the screen.

2 System Architecture

The application is organized into three panes. The navigation pane on the left lists document sections and page thumbnails. The content pane in the center displays the rendered pages. The AI pane on the right hosts a streaming chat interface backed by an OpenAI-compatible endpoint.

2.1 Rendering Pipeline

The rendering pipeline begins when the user selects a file. The binary is stored in memory as an `ArrayBuffer` and passed to `pdfjs-dist`. The library parses the file, creates a document proxy, and

exposes the number of pages. For each page that enters the viewport, we create a canvas element, request the page proxy, and call `render` with a scale derived from the current zoom level.

To support text selection, we render a transparent text layer over each canvas. The text layer is populated with absolutely positioned span elements whose locations and font sizes are computed from the page viewport. This preserves the visual fidelity of the canvas while allowing the browser to handle selection, copying, and screen readers.

2.2 Zoom and Scroll

Zoom is controlled through a reactive scale value in a Svelte rune store. When the user presses control and scrolls the mouse wheel, the scale is multiplied by a constant factor. A fit-to-width button computes the largest scale that fits the available client width for the first page. Scrolling is handled by a single overflow container that stacks pages vertically.

The user can also jump directly to a page by clicking an outline entry or a thumbnail. The target page element is scrolled into view with smooth behavior.

3 Language Model Integration

The AI pane submits the current document context together with the user’s prompt. We first extract the visible page text from the active text layers. This text is concatenated, truncated to fit the model context window, and sent as a system message. The user’s question is appended and streamed back token by token.

We deliberately avoid uploading the raw binary to any external service. All text extraction happens locally in the browser, so the document contents never leave the user’s device unless the user explicitly pastes them into the chat.

4 Evaluation

We measured rendering latency on a synthetic corpus of ten documents ranging from five to one hundred pages. Each document was generated with a mixture of paragraphs, equations, tables, and vector figures. Tests were run on a laptop with a four-core processor and eight gigabytes of memory.

Pages	First paint (ms)	Layout complete (ms)	Scroll fps
5	120	180	60
20	145	260	60
50	210	580	55
100	380	1120	48

The results in Table 1 show that first paint remains below four hundred milliseconds even for the longest document. Layout completion time grows linearly with page count because every canvas is

created upfront. Future work will investigate virtualized rendering to reduce initial layout time.

5 Related Work

Browser-based PDF rendering has been dominated by Mozilla's PDF.js since its release in 2011. PDF.js parses PDF byte streams, interprets page descriptions, and renders them to HTML5 canvas elements. Many readers wrap PDF.js in different frameworks but do not integrate language models.

Recent AI assistants for documents either require uploading files to a server or convert PDFs to markdown before processing. Our approach differs by keeping the rendering pipeline and the language model client fully local.

6 Conclusion

We described a reader architecture that renders PDFs faithfully in the browser while providing an AI assistant for navigation and summarization. The system achieves interactive frame rates on moderately long documents and keeps user data private by performing text extraction locally. Source code and benchmarks are available at <https://example.org/cimedaca>.

References

1. B. Jones and L. Smith. Adaptive layouts for responsive PDF readers. In Proceedings of the Web Systems Conference, pages 45-58, 2021.
2. A. Kapoor. Local first software: you own your data. The Local-First Journal, 2023.
3. J. Wang, K. Patel, and M. Chen. Evaluating text extraction quality from academic PDFs. Information Retrieval Letters, 19(3):210-225, 2022.